Herald

Herald — `pherald` Flavor Guide (Operator)

Field	Value
Revision	2
Created	2026-05-30
Last modified	2026-05-31
Status	active
Status summary	r2: documented in §5 (`listen`) that subscribers speak plain natural language — no command syntax — and the three-tier intent-resolution discipline (command-recognition fast-path → Claude Code intent inference → `clarify` reply-tag-and-ask fallback) per the contract `docs/design/INTENT_RECOGNITION.md`, with an explicit live-vs-contract note (Tier-2 dispatch is LIVE; Tier-1 recognizer + Tier-3 `clarify` are the contract being implemented). Prior r1: nano-detail operator reference for `pherald` (Project Herald) — the richest Herald flavor binary. Documents every subcommand surfaced by `pherald --help` (`serve`, `listen`, `watch`, `migrate`, `wizard`, `commit-push` + the §43 GitOps commands), the env/credentials each needs, real example invocations with the binary's actual flags, and which subcommands are live vs not-yet-implemented. ANTI-BLUFF: every claim below was derived from running the built `pherald` binary (`pherald --help`, `pherald <sub> --help`, `pherald version --json`) and reading `pherald/cmd/pherald/main.go` — nothing invented.
Issues	(none specific to this guide)
Continuation	Bump when `migrate down`/`migrate validate` land (currently not-yet-implemented), and when `pherald listen` / `pherald watch` capture operator-supplied live evidence under `docs/qa/`.

Table of contents

- §1. What `pherald` is
- §2. The subcommand surface
- §3. `version`
- §4. `serve` — the HTTP ingest daemon
- §5. `listen` — the inbound runtime
- §6. `watch` — workable-items SSoT → notify
- §7. `migrate` — schema migrations

- §8. `wizard credentials` — credential setup
- §9. `commit-push` + the §43 GitOps commands
- §10. Environment variable reference
- §11. References

---

§1. What `pherald` is

`pherald` is **Project Herald** — flavor key `p`, prefix `PHR`, default serving port **24791**. Per `commons/branding.go` its mission line is "Multi-mirror push + submodule propagation + project bindings", and it is the only flavor that carries the full event-ingest path plus the inbound runtime plus the GitOps command catalogue. It is the binary you reach for when a CI job, cron task, or AI agent needs to emit a CloudEvent, drive a Telegram conversation loop, or perform a §2-disciplined commit + multi-mirror push.

Build it (from a Herald checkout with the workspace and submodules initialised):

```
go build -o /tmp/pherald ./pherald/cmd/pherald
```

§2. The subcommand surface

The exact top-level surface, verbatim from `pherald --help`:

Subcommand	What it does	Live?
`commit-push`	Single-entrypoint locked commit + multi-mirror push (§2)	yes
`fetch-guard`	Pre-edit fetch + rebase enforcement (§11.4.37)	yes
`install-upstreams`	Configure mirror remotes from `upstreams/*.sh` declarations (§11.4.36)	yes
`listen`	Inbound runtime: Telegram `getUpdates` long-poll + Claude Code dispatch loop	yes
`migrate`	Apply or inspect `commons_storage` migrations (spec §9.6)	`up`/`status` live; `down`/`validate` not-yet-implemented
`pre-push`	Fetch + investigate + integrate hook (§11.4.71)	yes
`reopen`	Issues→Fixed reversal + Reopens history (§11.4.55)	yes
`serve`	Start the Project Herald HTTP server	yes
`submodule-propagate`	Owned-submodule walk in propagation order (§3)	yes
`version`	Print Project Herald version + build info	yes
`watch`	Watch the workable-items SSoT and notify channels on every change	yes
`wizard`	Interactive credential-setup wizards	yes
`completion`	Generate shell autocompletion (Cobra built-in)	yes

There are no top-level persistent flags beyond `-h/--help` and `-v/--version`.

§3. `version`

```
$ pherald version --json
{"arch":"arm64","binary":"pherald","build_time":"unknown","commit":"unknow
        dev"}
```

`version` (no flag) prints a human-readable banner; `--json` prints the machine-readable object above. `version`, `commit`, and `build_time` are `-ldflags`-injectable at build time (defaults are `0.0.0-dev` / `unknown` / `unknown`). `flavor` is always `p` for this binary.

§4. `serve` — the HTTP ingest daemon

`pherald serve` is the live HTTP ingest plane. The §32 7-stage Runner (parse → idempotency → tenant → policy → subscriber fan-out → channel dispatch → outcome record) hangs off `POST /v1/events`.

Flags (verbatim from `pherald serve --help`):

Flag	Meaning
`--http-port int`	TCP+UDP port to bind (default = flavor's DefaultPort, i.e. 24791)
`--tls-cert string`	Path to PEM-encoded TLS certificate (required when HTTP/3 is enabled, optional for TCP-only)
`--tls-key string`	Path to PEM-encoded TLS private key (paired with `--tls-cert`)
`--no-brotli`	Disable auto-wired Brotli compression middleware (useful for streaming routes that cannot be buffered)

Required environment (the serve plane fails loud at startup without these — by design, per the §107 anti-bluff posture documented at the top of `main.go`):

- `HERALD_PG_DSN` — `postgres://user:pass@host:port/dbname[?sslmode=disable]`. There is **no PG-less serve mode**; the Runner's idempotency archive + subscriber list + evidence writes all require Postgres.
- `HERALD_AUTH_MODE` — `hmac` or `jwks`. There is **no anonymous serve plane**; `POST /v1/events` is JWT-gated.
- `HERALD_AUTH_HMAC_SECRET` — required when `HERALD_AUTH_MODE=hmac` (32+ random bytes).

Optional environment:

- `HERALD_REDIS_URL` — enables the hot idempotency cache (and the JWKS-mode key cache). The Runner tolerates its absence (PG archive becomes the sole duplicate detector — degraded but functional).
- `HERALD_TGRAM_BOT_TOKEN` — registers the Telegram channel adapter. `null://` is always registered as the sandbox channel.

Example:

```
export HERALD_PG_DSN="postgres://herald:secret@127.0.0.1:24100/
        herald"
export HERALD_AUTH_MODE=hmac
```

```
export HERALD_AUTH_HMAC_SECRET="$(head -c32 /dev/urandom |
        base64)"
pherald serve --http-port 24791
```

`/v1/healthz`, `/v1/readyz`, and `/metrics` come up immediately and bypass JWT (K8s-probe friendly). `SIGINT`/`SIGTERM` triggers a graceful shutdown that drains in-flight requests and closes the PG/Redis pools. For the full first-CloudEvent walkthrough see `docs/INTEGRATION.md` §7–§9.

§5. `listen` — the inbound runtime

`pherald listen` is the long-running inbound loop (Wave 6). It wires `tgram.Subscribe` (Telegram `getUpdates` long-poll) to the production `inbound.Dispatcher`: every inbound message is dispatched through Claude Code (Opus, pinned) per the §32 inbound pipeline, and the parsed `<<<HERALD-REPLY>>>` block is routed by `action` (`reply` / `issue.open` / `event.emit` / item mutations).

Subscribers speak plain natural language — no command syntax. Subscribers writing to the bot do NOT need to know any command syntax: there is no `COMMAND:` prefix and no fixed grammar. They send a clear message in their own words and the System determines the intent. Intent resolution is the three-tier discipline defined in the contract [`docs/design/INTENT_RECOGNITION.md`]: (1) a deterministic command-recognition fast-path for clear imperatives; (2) Claude Code intent inference (the LLM-driven dispatch this `listen` loop already runs) when no command matches; (3) a `clarify` fallback — when the intent cannot be determined, the System replies to the original message, @-tags the sender, and asks a precise clarifying question naming the candidate intents (never guessing an action, never silently dropping a message). The recognized command set (close/assign/status-change → `item.update`, "open a bug/task: …" → `issue.open`, "investigate ATM-N" → `investigation.start`, questions → `reply`) and the `clarify` action are specified in that contract; see also `docs/guides/MESSENGER_CHANNELS.md` §6B.

> The Tier-2 Claude Code dispatch + `<<<HERALD-REPLY>>>` action routing described here are LIVE in this `listen` loop today; the deterministic Tier-1 `CommandRecognizer` and the Tier-3 `clarify` action are the contract Herald is implementing — `docs/design/INTENT_RECOGNITION.md` is the authoritative spec, not a claim that the fast-path is already wired into the binary.

Required environment (verbatim from `pherald listen --help`):

- `HERALD_TGRAM_BOT_TOKEN` — Telegram bot token (validated via `getMe` on boot).
- `HERALD_TGRAM_CHAT_ID` — numeric chat ID; included in the `tgram://` URL.

Optional environment:

- `HERALD_PROJECT_NAME` — Claude Code session name (else `basename(cwd)` else `Herald`).
- `HERALD_CLAUDE_BIN` — path to the `claude` CLI (default: `$PATH` lookup).

Flags:

Flag	Meaning
`--qa-out-dir string`	Journal every inbound/CC/outbound event to `<dir>/transcript.jsonl` and copy attachments to `<dir>/attachments/<sha256>.<ext>` — the Wave 6 T10a §107.x evidence primitive.
`--docs-dir string`	Docs root containing `Issues.md` / `Fixed.md` / `Status.md` / `CONTINUATION.md` / `Help.md`. Default `docs`. `Issues.md` **MUST exist — startup fails otherwise** (Wave 6.5 T7 §107 fail-loud).
`--db string`	Workable-items SQLite DB path backing `item.update`/`item.delete`/ confirmed-investigation actions (default `$HERALD_WORKABLE_DB` or `docs/workable_items.db`; HRD-152). An empty path with no env makes those actions return an explicit "no ItemMutator configured" error.

Signal handling: `SIGINT`/`SIGTERM` cancels the long-poll cleanly via `signal.NotifyContext`.

```
export HERALD_TGRAM_BOT_TOKEN=8000000000:XXXX
export HERALD_TGRAM_CHAT_ID=987654321
pherald listen --qa-out-dir docs/qa/manual-run-$(date +%s)
```

> NOTE: `listen` requires real Telegram credentials and a `claude` CLI to do anything useful end-to-end; without them it fails loud at boot. Fully-autonomous (no-human) testing of this loop is the `qaherald` flavor's job — see `docs/guides/QAHERALD.md`.

§6. `watch` — workable-items SSoT → notify

`pherald watch` is a second long-running daemon (HRD-153, ATMOSphere integration). It composes the workable-items single-source-of-truth **watch → diff → notify** flow:

1. Opens the shared workable-items SQLite DB (`--db`) and snapshots every item at the watched locations (Issues + Fixed).
2. Starts a `commons_watch.Watcher` (fsnotify + WAL-poll) on the DB file and the Markdown trackers (`--issues` / `--fixed`).
3. On every change: re-lists current items, diffs against the prior snapshot, renders each per-property delta, and fans it out through the production `ChannelDispatcher` to every configured recipient's channel.

Channel/recipient config reuses the same `HERALD_CHANNELS` + per-channel namespaced env as `pherald listen` (the production fan-out path) — see `docs/guides/MESSENGER_CHANNELS.md`.

Flags:

Flag	Default	Meaning
`--db string`	`$HERALD_WORKABLE_DB` or `docs/workable_items.db`	Workable-items SQLite DB path
`--issues string`	`docs/Issues.md`	Issues.md tracker path (watched)
`--fixed string`	`docs/Fixed.md`	Fixed.md tracker path (watched)
	`1s`	

Flag	Default	Meaning
`--poll duration`		WAL-poll fallback interval (`0` disables, fsnotify only)

```
pherald watch --db docs/workable_items.db --poll 1s
```

`SIGINT`/`SIGTERM` cancels the watch loop cleanly. The watcher mechanics are documented in `docs/guides/COMMONS_WATCH.md`; the store in `docs/guides/COMMONS_WORKABLE.md`.

§7. `migrate` — schema migrations

`pherald migrate` runs Herald's embedded SQL migrations against Postgres. It requires `HERALD_PG_DSN` (no silent default).

Subcommand	State	Behaviour
`up`	live	Apply every pending migration in version order.
`status`	live	Report the highest applied migration version.
`down`	**not yet implemented**	Roll back the most recent migration (destructive op per §9.1, future HRD).
`validate`	**not yet implemented**	Audit applied migrations vs the embedded set (schema-drift detection, future HRD).

```
export HERALD_PG_DSN="postgres://herald:secret@127.0.0.1:24100/
        herald"
pherald migrate up
pherald migrate status     # e.g. "schema version: 12"
```

`down` and `validate` exist as subcommands but return a not-yet-implemented message — do not script against them yet.

§8. `wizard credentials` — credential setup

`pherald wizard credentials [service]` is the canonical credential-setup tool. Supported services (LIVE): `telegram` (HRD-011 — bot token + chat ID, validated via `getMe` + `getChat`), `claude-code` (HRD-012 — `claude` binary + project name, validated via `--version`), and `all` (both, in order). Without an argument it prompts from a menu (interactive only).

Resolution order is **flag → env → prompt**. Flags:

Flag / env	Service
`--bot-token` / `HERALD_TGRAM_BOT_TOKEN`	Telegram
`--chat-id` / `HERALD_TGRAM_CHAT_ID`	Telegram (skips `getUpdates` polling)
`--claude-bin` / `HERALD_CLAUDE_BIN`	Claude Code
`--claude-project` / `HERALD_CLAUDE_PROJECT_NAME`	Claude Code

Flag / env	Service
`--claude-session-uuid` / `HERALD_CLAUDE_SESSION_UUID`	Claude Code
`--non-interactive`	Fail loud if a required value is missing — no prompts
`--shell-target=zshrc|bashrc|profile`	Override the shell-startup-file picker

Per Universal §11.4.10 the wizard never prints raw credentials back, masks values in the `~/.herald/credentials.md` summary, and re-reads the shell-startup file to verify the write. Env-supplied values are never re-persisted (they're already in your shell file).

```
pherald wizard credentials telegram \
    --bot-token=8000000000:XXXX --chat-id=987654321 \
    --shell-target=zshrc --non-interactive
```

Detailed per-service walkthroughs: `docs/guides/messengers/TELEGRAM.md`, `docs/guides/dispatchers/CLAUDE_CODE.md`, `docs/guides/OPERATOR_CREDENTIALS.md`.

§9. `commit-push` + the §43 GitOps commands

`pherald commit-push` performs a §2-disciplined commit through a single locked entrypoint (an `O_CREAT|O_EXCL` commit-lock under `.git/`) and, with `--push`, fans the commit out to every configured mirror remote. It wraps the canonical constitution `commit_all.sh` when discoverable.

Flag	Meaning
`-m, --message string`	Commit message (**required**)
`--push`	Fan the commit out to every configured mirror remote
`--scope strings`	Paths to stage (default: all tracked changes)
`--repo string`	Repo dir (default: discovered from CWD)
`--emit`	Also drive the §2 verdict as a real constitution event

The remaining §43 project-lifecycle GitOps commands surfaced by `pherald --help` — `fetch-guard` (§11.4.37 pre-edit fetch+rebase), `install-upstreams` (§11.4.36 mirror-remote config from `upstreams/*.sh`), `pre-push` (§11.4.71 fetch+investigate+integrate hook), `reopen` (§11.4.55 Issues→Fixed reversal), `submodule-propagate` (§3 owned-submodule walk) — are all wired (v1.0.0 Batch C, HRD-029/030/043/044/049/053). Run `pherald <command> --help` for each one's flag surface.

§10. Environment variable reference

Variable	Used by	Required?
`HERALD_PG_DSN`	`serve`, `migrate`	required for both
`HERALD_AUTH_MODE`	`serve`	required (`hmac`/`jwks`)
`HERALD_AUTH_HMAC_SECRET`	`serve`	required when `HERALD_AUTH_MODE=hmac`
`HERALD_REDIS_URL`	`serve`	optional (enables hot idempotency cache)

Variable	Used by	Required?
HERALD_TGRAM_BOT_TOKEN	serve (channel), listen, wizard	required for listen
HERALD_TGRAM_CHAT_ID	listen, wizard	required for listen
HERALD_PROJECT_NAME	listen (CC session name)	optional
HERALD_CLAUDE_BIN	listen, wizard	optional
HERALD_WORKABLE_DB	listen, watch	optional (DB path default)
HERALD_CHANNELS + per-channel env	watch, listen (fan-out)	per channel config

Resolution order project-wide: exported shell vars win; .env is fallback only.

§11. References

- Source: pherald/cmd/pherald/main.go (+ serve.go, listen.go, watch.go, migrate.go, wizard*.go, gitops_cmds.go, stubs.go).
- Branding: commons/branding.go (flavor p).
- Shared CLI scaffold: commons/cli/ (NewRootCmd, VersionCmd).
- Integration walkthrough: docs/INTEGRATION.md (§5–§9 cover serve + first CloudEvent).
- Credentials: docs/guides/OPERATOR_CREDENTIALS.md, docs/guides/messengers/TELEGRAM.md, docs/guides/dispatchers/CLAUDE_CODE.md.
- Companion flavor guides: docs/guides/{SHERALD,CHERALD,BHERALD,RHERALD,IHERALD,SCHERALD,QAHERALD}.md.

Sources verified

Verified 2026-05-30: internal doc — no external online sources. Every subcommand, flag, default, env var, and "live vs not-yet-implemented" claim above was derived by **running the built** pherald **binary** (pherald --help; pherald serve|listen|watch|migrate|wizard|wizard credentials|commit-push --help; pherald version --json) and reading pherald/cmd/pherald/main.go + commons/branding.go on 2026-05-30. No flags were invented. Re-verify whenever the pherald Cobra surface changes (new subcommand, flag rename, or migrate down/validate landing).